Deep Learning Fundamentals

Week 5 – AI Internship | Task 1 | Deep Learning & Neural Networks

1. What is Deep Learning?

Deep Learning is a part of Machine Learning that uses **artificial neural networks with many layers** to learn patterns from data. Instead of a person writing rules, the network learns the rules by looking at a large number of examples. It is used in image recognition, voice assistants, language translation and self-driving cars. The word 'deep' comes from the many hidden layers in the network.

2. AI vs Machine Learning vs Deep Learning

	Artificial Intelligence	Machine Learning	Deep Learning
Meaning	Machines that act intelligent	AI that learns from data	ML that uses multi-layer neural networks
Relation	The biggest field	A subset of AI	A subset of ML
Data needed	Can work with rules only	Small to medium data	Large amounts of data
Example	Chess program, chatbot	Spam filter, price prediction	Face recognition, digit reading

3. What is an Artificial Neural Network (ANN)?

An ANN is a model inspired by the human brain. It is made of many small units called neurons arranged in layers and connected to each other. Data goes in, passes through the layers, and a prediction comes out. The network learns by changing the strength of its connections.

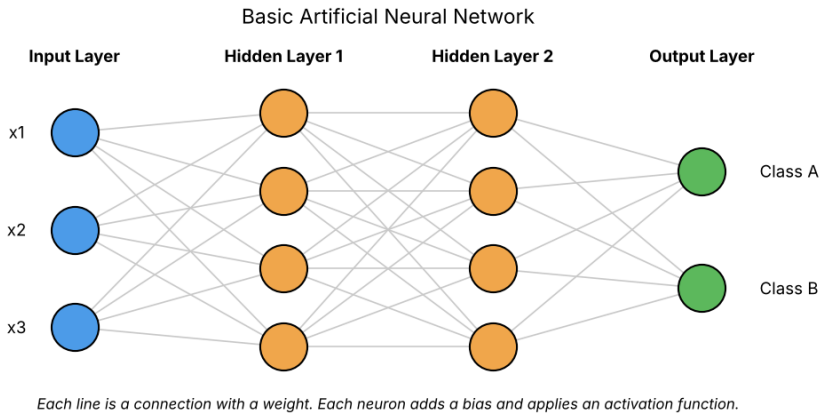


Figure 1: A basic neural network with an input layer, two hidden layers and an output layer

4. What is a Neuron?

A neuron is the basic unit of a neural network. It receives inputs, multiplies each input by a weight, adds them together with a bias, and then passes the result through an activation function to produce its output.

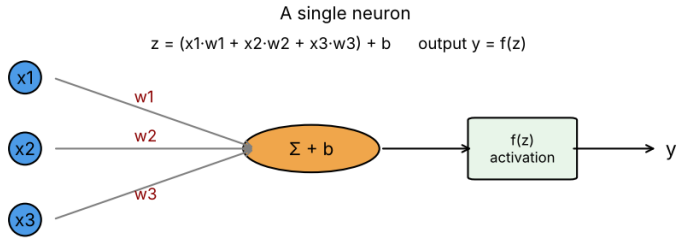


Figure 2: Working of a single neuron

5. Layers of a Neural Network

Input Layer: receives the raw data. It has one neuron for each feature (for example 4 neurons for the 4 measurements of an Iris flower). It does no calculation.

Hidden Layer: sits between input and output. It does the real learning by finding patterns in the data. More hidden layers let the network learn more complex patterns.

Output Layer: gives the final answer. For a classification problem it has one neuron per class (for example 3 neurons for 3 flower species).

6. Weights and Bias

Weights decide how important each input is. A large weight means that input has a big effect on the result. **Bias** is an extra number added to the neuron that lets it shift its output up or down, so the neuron can still be activated even when all inputs are zero. The weights and biases are the values that the network learns during training.

7. Activation Functions

An activation function decides the output of a neuron and adds **non-linearity**, so the network can learn complex patterns and not only straight lines. **Sigmoid** squeezes values between 0 and 1 (used for yes/no outputs). **ReLU** returns 0 for negative values and the same value for positive values (the most popular for hidden layers because it is simple and fast). **Tanh** squeezes values between -1 and 1. **Softmax** turns the outputs into probabilities that add up to 1 (used in the output layer for multi-class problems).

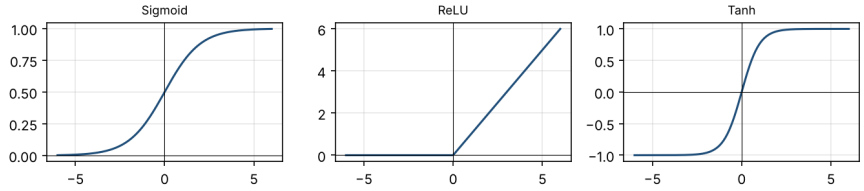


Figure 3: Common activation functions

8. Epoch, Loss and Optimizer

Epoch: one full pass of the whole training dataset through the network. Training usually needs many epochs, because the network improves a little in each one.

Loss: a number that tells how wrong the prediction is. A high loss means bad predictions and a low loss means good predictions. Examples are cross-entropy loss for classification and mean squared error for regression. The aim of training is to make the loss as small as possible.

Optimizer: the algorithm that updates the weights and biases to reduce the loss. It uses the gradients from backpropagation. Common optimizers are SGD (Stochastic Gradient Descent) and Adam. Adam is popular because it adjusts the learning rate by itself and trains fast.

9. Basic Working Flow

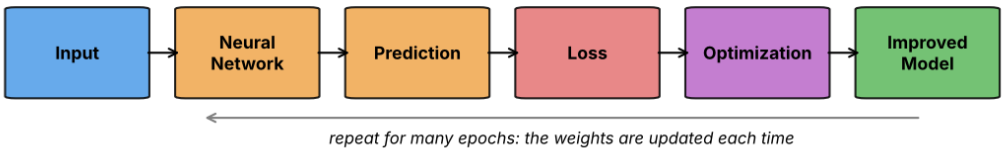


Figure 4: Training flow: Input → Neural Network → Prediction → Loss → Optimization → Improved Model

Step 1 – Input: the data (for example the 4 measurements of a flower) is fed into the input layer.

Step 2 – Neural Network: the data moves forward through the hidden layers. Each neuron multiplies by weights, adds bias and applies the activation function. This is called *forward propagation*.

Step 3 – Prediction: the output layer gives its prediction, for example 'Setosa'.

Step 4 – Loss: the prediction is compared with the real answer and the loss function calculates the error.

Step 5 – Optimization: the error is sent backwards through the network (*backpropagation*) and the optimizer changes the weights and biases a little to reduce the error.

Step 6 – Improved Model: the model is now slightly better. This cycle repeats for every batch of data and for many epochs until the loss becomes small and accuracy becomes high.

References: TensorFlow / Keras documentation (keras.io); scikit-learn documentation.